Proposal Introduction

In the field of emergency management, the "Golden Hour" refers to the critical window where medical or professional intervention significantly increases the probability of survival and minimizes long-term damage. Despite advancements in general-purpose navigation technology, existing platforms are often optimized for commercial efficiency or consumer convenience, which may not prioritize the unique urgency required by emergency services.

Finding the Shortest Path is an integrated emergency response and routing system designed to bridge the gap between incident reporting and responder deployment. By modeling urban road networks as a specialized data graph, the system focuses on identifying and visualizing the most efficient corridors for crisis intervention. The platform provides a synchronized environment where victims can use SOS signals, allowing the system to calculate and present optimized route data to the needed emergency establishment. By prioritizing calculated travel time over simple distance, the system aims to provide a reliable framework for hospitals, police departments, and evacuation centers to improve their operational planning across complex city landscapes.

Objectives

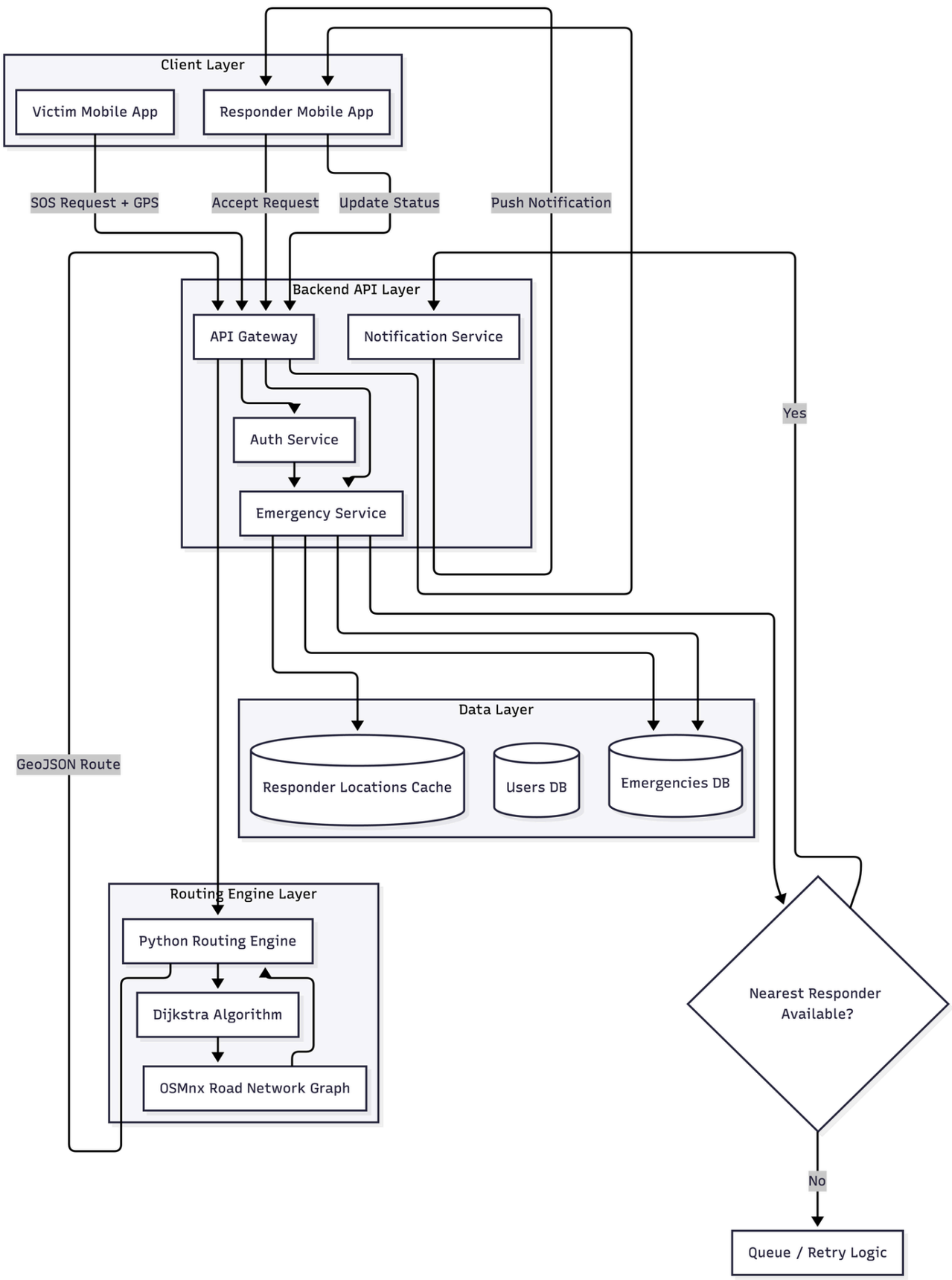
1.1 General Objective

To design, develop, and implement a routing and management system that facilitates rapid communication and identifies the most time-efficient paths for emergency service providers.

2.2: Specific Objectives

1. **Incident Reporting:** To establish a synchronized communication channel that ensures the accurate transmission of location data from users to the centralized system.
2. **Network Modeling:** To construct a digital representation of real-world road networks that prioritizes transit time as the primary variable for path selection.
3. **Responder Allocation:** To design a backend logic that identifies and alerts the most suitable responder based on proximity and availability within the system's data layer.
4. **Data Management:** To develop a database architecture to track and manage the complete lifecycle of an emergency request from initiation to resolution.
5. **Route Visualization:** To generate precise graphical representations of the shortest calculated paths to assist responders in identifying efficient transit corridors.

Logic Flow Diagram



Program Listing

Routing Engine (Python – Core System Focus)

The Python Routing Engine is the core intelligence of the system. It is responsible for computing the fastest possible route between the emergency location and the assigned responder using real-world road data.

Unlike basic navigation systems that rely on distance, this engine optimizes for travel time, making it more suitable for emergency response scenarios where speed is critical.

Technology	Purpose
Flask	Lightweight API server to handle routing requests
OSMnx	Extracts and models real-world road networks
NetworkX	Executes graph algorithms (Dijkstra)
GeoJSON	Standard format for map visualization

Graph-Based Routing

The road network is modeled as a **graph**:

- **Nodes (Vertices):** Intersections or points on the map
- **Edges:** Roads connecting nodes
- **Weights:** Travel time for each road segment

This allows the system to mathematically determine the optimal path.

Algorithm Used

Dijkstra's Algorithm

The system uses Dijkstra's algorithm to compute the **minimum-cost path** between two nodes.

Shortest Path: min∑i=1nwi\text{Shortest Path: } \min \sum_{i=1}^{n} w_iShortest Path: min∑i=1nwi

Data Dictionary

Table: users

Column	Type	Description
`id`	BigInt (PK)	Unique identifier for each user
`name`	String	Full name of the user
`email`	String (Unique)	User's email address for authentication
`email_verified_at`	Timestamp (Nullable)	Date and time when email was verified
`password`	String	Hashed password
`remember_token`	String (Nullable)	Token for session persistence
`created_at`	Timestamp	Record creation time
`updated_at`	Timestamp	Last update time

Table: respondents

Column	Type	Description
`id`	BigInt (PK)	Unique identifier for each responder
`name`	String	Full name of the responder
`email`	String (Unique)	Email used for login/authentication
`password`	String	Hashed password
`phone`	String (Nullable)	Contact number
`base_name`	String	Name of assigned base (default: QC Ambulance Base)
`base_latitude`	Decimal(10,8)	Latitude of responder's base location
`base_longitude`	Decimal(11,8)	Longitude of responder's base location
`is_active`	Boolean	Indicates if responder is available
`created_at`	Timestamp	Record creation time

updated_at	Timestamp	Last update time

Table: emergencies

Column	Type	Description
id	BigInt (PK)	Unique identifier for each emergency
user_id	BigInt (FK, Nullable)	References users.id (victim)
respondent_id	BigInt (FK, Nullable)	References respondents.id
type	String	Type of emergency (e.g., Fire, Medical)
latitude	Decimal(10,8)	GPS latitude of incident
longitude	Decimal(11,8)	GPS longitude of incident
status	String	Status (active, accepted, resolved)
created_at	Timestamp	Record creation time
updated_at	Timestamp	Last update time

Table: password_reset_tokens

Column	Type	Description
email	String (PK)	Email associated with reset request
token	String	Password reset token
created_at	Timestamp (Nullable)	Time token was created

Table: sessions

Column	Type	Description
id	String (PK)	Unique session identifier
user_id	BigInt (FK, Nullable)	Associated user
ip_address	String (45)	IP address of client
user_agent	Text	Browser/device info
payload	LongText	Session data
last_activity	Integer	Last activity timestamp

Table: cache

Column	Type	Description
`key`	String (PK)	Unique cache key
`value`	MediumText	Cached data
`expiration`	Integer	Expiration timestamp

Table: cache_locks

Column	Type	Description
`key`	String (PK)	Lock identifier
`owner`	String	Owner of the lock
`expiration`	Integer	Lock expiration time

Table: personal_access_tokens

Column	Type	Description
`id`	BigInt (PK)	Unique token ID
`tokenable_type`	String	Model type (User/Respondent)
`tokenable_id`	BigInt	ID of related model
`name`	Text	Token name/label
`token`	String (64, Unique)	API token value
`abilities`	Text (Nullable)	Permissions of the token
`last_used_at`	Timestamp (Nullable)	Last usage time
`expires_at`	Timestamp (Nullable)	Expiration time
`created_at`	Timestamp	Record creation time
`updated_at`	Timestamp	Last update time